

Tee Split — Build Spec

Product: A simple, no-login golf buddy-trip tool for splitting costs, tracking side bets, and keeping hole-by-hole scores. **Audience:** Casual golfers organizing an annual/occasional buddies trip — not serious tournament players. **Domain:** teesplit.us **Reference**

implementation: `tee-split.jsx` (working React prototype with full UI, business logic, and interaction flows already validated). Treat it as the source of truth for UX and calculations — this spec describes how to port it into a real hosted app, not redesign it.

1. Tech Stack

- **Framework:** Next.js (App Router) — single project for frontend + API routes
 - **Database:** Supabase (Postgres) — free tier is sufficient at this scale
 - **Hosting:** Vercel, connected to teesplit.us
 - **No auth for v1** — trips are accessed via a unique code/link, not user accounts. Anyone with the link can view and edit. (This matches the prototype and keeps friction at zero — revisit if abuse becomes a problem.)
-

2. Data Schema

```
-- Trips
trips (
  id          uuid primary key default gen_random_uuid(),
  code        text unique not null,          -- 5-char invite code, e.g. "ABCDE"
  name        text not null,
  start_date  date,
  end_date    date,
  created_at  timestamptz default now()
)

-- Players (no accounts — just names scoped to a trip)
players (
  id          uuid primary key default gen_random_uuid(),
  trip_id     uuid references trips(id) on delete cascade,
  name        text not null,
```

```

    created_at  timestampz default now()
)

-- Expenses
expenses (
    id          uuid primary key default gen_random_uuid(),
    trip_id     uuid references trips(id) on delete cascade,
    description  text not null,
    amount      numeric(10,2) not null,
    paid_by     uuid references players(id),
    created_at  timestampz default now()
)

expense_splits (          -- who the expense is split between (many-to-many)
    expense_id  uuid references expenses(id) on delete cascade,
    player_id   uuid references players(id) on delete cascade,
    primary key (expense_id, player_id)
)

-- Rounds (a round of golf played during the trip)
rounds (
    id          uuid primary key default gen_random_uuid(),
    trip_id     uuid references trips(id) on delete cascade,
    name        text not null,          -- course name
    date        date,
    num_holes   int not null check (num_holes in (9, 18)),
    par         int[] not null,         -- length == num_holes
    created_at  timestampz default now()
)

round_scores (
    round_id    uuid references rounds(id) on delete cascade,
    player_id   uuid references players(id) on delete cascade,
    strokes     int[] not null,         -- length == num_holes, null entries allowed
    primary key (round_id, player_id)
)

-- Side bets (skins, Nassau, greenies, etc.) tied to a round
sidebets (
    id          uuid primary key default gen_random_uuid(),
    trip_id     uuid references trips(id) on delete cascade,
    round_id    uuid references rounds(id) on delete cascade,
    name        text not null,
    entry_amount numeric(10,2) not null,
    created_at  timestampz default now()
)

```

```

sidebet_participants (
  sidebet_id  uuid references sidebets(id) on delete cascade,
  player_id   uuid references players(id) on delete cascade,
  primary key (sidebet_id, player_id)
)

sidebet_winners (
  sidebet_id  uuid references sidebets(id) on delete cascade,
  player_id   uuid references players(id) on delete cascade,
  primary key (sidebet_id, player_id)
)

```

Notes for the port:

- The prototype stores `par` and `strokes` as JS arrays inside a single JSON blob. Postgres native arrays (`int[]`) work fine and keep querying simple — no need for a separate `holes` table.
- Cascade deletes so removing a trip/player cleans up everything downstream (matches prototype behavior when removing a player).

3. API Routes

All scoped under `/api/trips/[code]/...`. No auth middleware for v1 — the code itself is the access control.

Method	Route	Purpose
POST	<code>/api/trips</code>	Create a trip. Generates a unique code. Returns trip object.
GET	<code>/api/trips/[code]</code>	Fetch full trip state (players, expenses, rounds, sidebets) — the prototype currently loads this as one blob; keep that pattern, one call hydrates the whole app.
PATCH	<code>/api/trips/[code]</code>	Update trip name/dates.
POST	<code>/api/trips/[code]/players</code>	Add a player.
DELETE	<code>/api/trips/[code]/players/[id]</code>	Remove a player (also strips them from any expense splits /

		sidebets, matching prototype's <code>removePlayer</code> behavior).
POST	<code>/api/trips/[code]/expenses</code>	Add an expense (description, amount, paidBy, splitWith[]).
DELETE	<code>/api/trips/[code]/expenses/[id]</code>	Remove an expense.
POST	<code>/api/trips/[code]/rounds</code>	Add a round (name, date, numHoles). Initializes par array to all-4s.
PATCH	<code>/api/trips/[code]/rounds/[id]/par</code>	Update a single hole's par.
PATCH	<code>/api/trips/[code]/rounds/[id]/scores</code>	Update a single player's score for a single hole.
DELETE	<code>/api/trips/[code]/rounds/[id]</code>	Remove a round (cascades sidebets tied to it).
POST	<code>/api/trips/[code]/sidebets</code>	Add a side bet (roundId, name, entryAmount, participants[], winners[]).
DELETE	<code>/api/trips/[code]/sidebets/[id]</code>	Remove a side bet.

Balance/settle-up logic stays client-side or as a computed (non-persisted) response

— port `computeBalances()` and `settleUp()` directly from the prototype almost unchanged; they're pure functions over the trip data, no schema changes needed.

4. Screens

Matches the prototype's tab structure. Build as pages/routes under `/trip/[code]/...` so each screen is a real shareable URL (this also sets up deep-linking later, e.g. sharing a specific round).

1. **Landing (/)** — Create trip form + join-by-code form. On create/join, redirect to `/trip/[code]`.
2. **Trip Home (/trip/[code])** — Player list, add/remove players, trip name/dates, copy invite link button.
3. **Costs (/trip/[code]/expenses)** — Add expense form (description, amount, paid by,

split-with multi-select), expense list.

4. **Scores (/trip/[code]/scores)** — Round list with quick leaderboard preview. Tapping a round goes to:
 - **Round detail (/trip/[code]/scores/[roundId])** — Full hole-by-hole scorecard grid (editable par row + per-player score entry), round leaderboard, side bets section (add bet, list existing bets with pot/payout).
5. **Settle Up (/trip/[code]/settle)** — Net balance per player, minimum-transaction payout list.
6. **Recap (/trip/[code]/recap)** — Total spent, rounds played, trip-long stroke leaderboard, final settle-up, “Copy recap to share” button.

Invite link (replaces manual code entry): `teesplit.us/trip/[code]` should work as a direct shareable link — anyone opening it lands straight in the trip, no code re-entry needed. This is the #1 UX fix over the prototype.

5. Business Logic to Port As-Is

These are already correct in the prototype — copy the logic, don’t redesign it:

- `computeBalances(trip)` — nets out expenses and sidebets per player into a single balance.
 - `settleUp(balances)` — greedy debtor/creditor matching to produce the minimum number of payment transactions.
 - `relScore(total, par)` — formats a score relative to par (`E` , `+2` , `-1`).
 - Sidebet payout math: $\text{pot} = \text{entryAmount} \times \text{participants.length}$; $\text{payout} = \text{pot} / \text{winners.length}$, split evenly on ties.
-

6. Out of Scope for v1 (explicitly deferred)

- User accounts / auth
- Payments (Stripe/Venmo integration) — pricing model still being validated manually
- Trip pass paywall / feature gating — revisit once pricing is validated with real users
- Packing checklists, photo logs, weather integration — parked feature ideas, not

committed

- Any live/hole-by-hole scoring *during* a round (real-time sync) — the prototype's model is post-hoc entry, keep it that way for v1
-

7. Definition of Done for v1

- Trip creation/join works via real shareable link
- All CRUD from the prototype (players, expenses, rounds/scores, sidebets) ported with no functional regressions
- Settle Up and Recap match the prototype's calculations exactly
- Deployed and reachable at teesplit.us
- Mobile-responsive (primary usage is on-course, on a phone)